МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ

РОССИЙСКОЙ ФЕДЕРАЦИИ Федеральное государственное бюджетное образовательное учреждение высшего образования «РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ Г.В.ПЛЕХАНОВА»

Техникум Пермского института (филиала)

Практическая работа №2

по дисциплине: Поддержка и тестирование программных модулей

на тему: Написание программы «Калькулятор» с использованием WPF

Руководитель

Д.Б. Берестов/ И.О. Фамилия/

«20» марта 2026 г.

Исполнитель

В.Е. Оборина/ И.О. Фамилия /

Группа ИПо-22

«20» марта 2026 г.

Пермь, 2026

Оглавление

Цель практической работы 3
Структура проекта 3
Создание проекта в Visual Stydio 4
Создание Библиотеки Классов 5
Создание проекта тестов 6
Подключение библиотеки к WPF 7
Создание пользовательского интерфейса 8
Подключение модульного теста к библиотеке 10
Код для тестов и его запуск 11
Заключение 13
Основные цели задания достигнуты 13

Цель практической работы

Целью этой работы является создание приложения *«Калькулятор»* на базе *WPF* в *Visual Studio*. Главной задачей стало разделение программы на части: интерфейс пользователя и отдельную библиотеку классов, где находятся все математические расчеты. В ходе выполнения задания нужно было реализовать основные действия, включая возведение в степень, а также настроить обработку ошибок. Основываясь на опыте первой практики, я добавила проверку деления на ноль, чтобы программа не закрывалась с ошибкой, а выдавала понятное сообщение пользователю. Также важной частью работы стало написание модульных тестов, которые автоматически проверяют правильность работы всех функций в библиотеке, чтобы калькулятор всегда считал верно и был надежным при использовании.

Структура проекта

- WPF-приложение – это внешняя часть калькулятора с кнопками и экраном. Оно нужно для того, чтобы пользователь мог нажимать на цифры, вводить примеры и видеть готовый ответ.
- Библиотека классов – это «мозги» программы. Здесь нет кнопок, а хранится только код для математических расчетов. Мы вынесли его отдельно, чтобы логика вычислений не зависела от дизайна.
- Модульные тесты – это автоматические проверки кода. Они сами тестируют нашу библиотеку и сразу показывают, если в расчетах есть ошибка (например, при делении на ноль или возведении в степень).

Создание проекта в Visual Stydio

1. Запускаем *Visual Studio* и создаем проект.
2. Выбираем шаблон *«Приложение WPF (.NET Framework)»*
3. Называем проект *«oborina_calculator_2pr» (рис.1)*

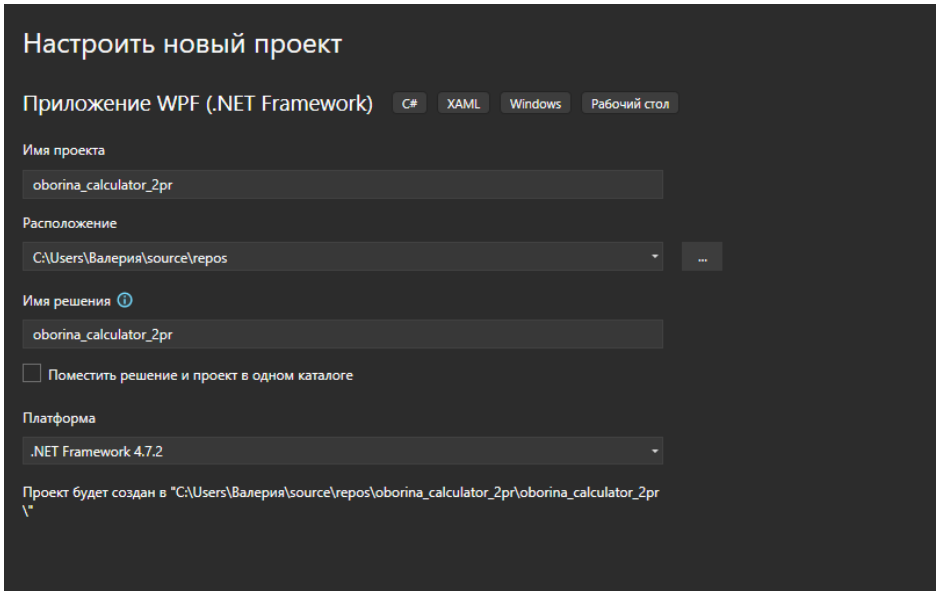

Рис.1 – Создание проекта

Создание Библиотеки Классов

1. В обозревателе решений нажимаем ПКМ по решению - добавить - создать проект
2. Выбираем шаблон *«Библиотека классов (.NET Framework)»*
3. Называем проект *«oborina_library1»*
4. Нажимаем кнопку *«Создать»*
5. В классе *Calc.cs* реализуем основные арифметические операции: сложение, вычитание, умножение, деление и возведение в степень.
6. Опираясь на опыт первой практики, я добавила ручную проверку деления на ноль внутри метода `Div`. Если делитель равен 0, программа генерирует исключение `DivideByZeroException`. Это гарантирует, что ошибка будет обработана на уровне логики, а не интерфейса. *(рис. 2)*

```
ina_library1                                   OborinaLibrary1.Calc
using System;

namespace OborinaLibrary1
{
    Ссылок: 6
    public static class Calc
    {
        Ссылок: 1
        public static double Add(double a, double b) => a + b;

        Ссылок: 1
        public static double Sub(double a, double b) => a - b;

        Ссылок: 1
        public static double Mul(double a, double b) => a * b;

        Ссылок: 2
        public static double Div(double a, double b)
        {
            if (b == 0)
            throw new DivideByZeroException("Деление на ноль невозможно");
            return a / b;
        }

        Ссылок: 1
        public static double Pow(double a, double b) => Math.Pow(a, b);
    }
}
```

Рис.2 – код класса Calc.cs

Создание проекта тестов

1. Нажимаем ПКМ по решению - добавить - создать проект.
2. Выбираем шаблон *«Проект модульного теста (.NET Framework)»*.
3. Называем проект *«oborina_tests» (рис.3)*

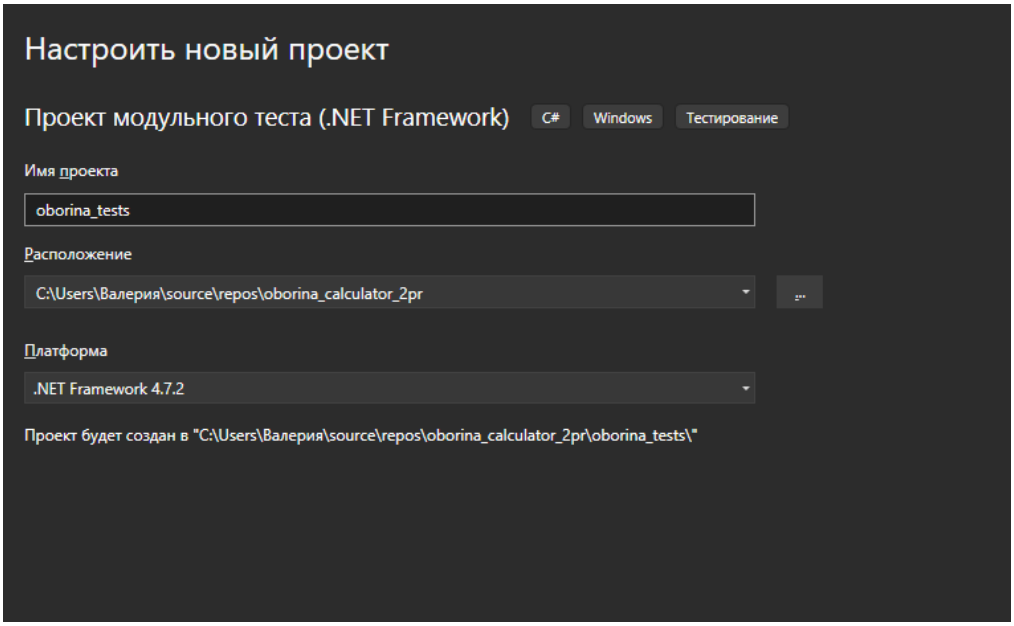

Рис.3 – Создание проекта модульного теста

Создание отдельного проекта для модульного тестирования является обязательным стандартом в современной разработке программного обеспечения. Мы выбираем шаблон «Проект модульного теста», чтобы иметь возможность автоматически проверять каждый метод нашей библиотеки классов. Основное преимущество такого подхода заключается в том, что тесты запускаются изолированно от графического интерфейса. Это позволяет быстро найти ошибку в математической логике, не запуская сам калькулятор вручную. Наличие отдельного тестового модуля в решении гарантирует, что любые будущие изменения в коде не нарушат правильность уже работающих функций, таких как возведение в степень или деление чисел.

Подключение библиотеки к WPF

1. В обозревателе решений нажимаем ПКМ по проекту *oborina_calculator_2pr* - добавить - ссылка.
2. Во вкладке проекты ставим галочку на проекте *oborina_library1*.
3. Нажимаем кнопку *«ОК» (рис.4)*

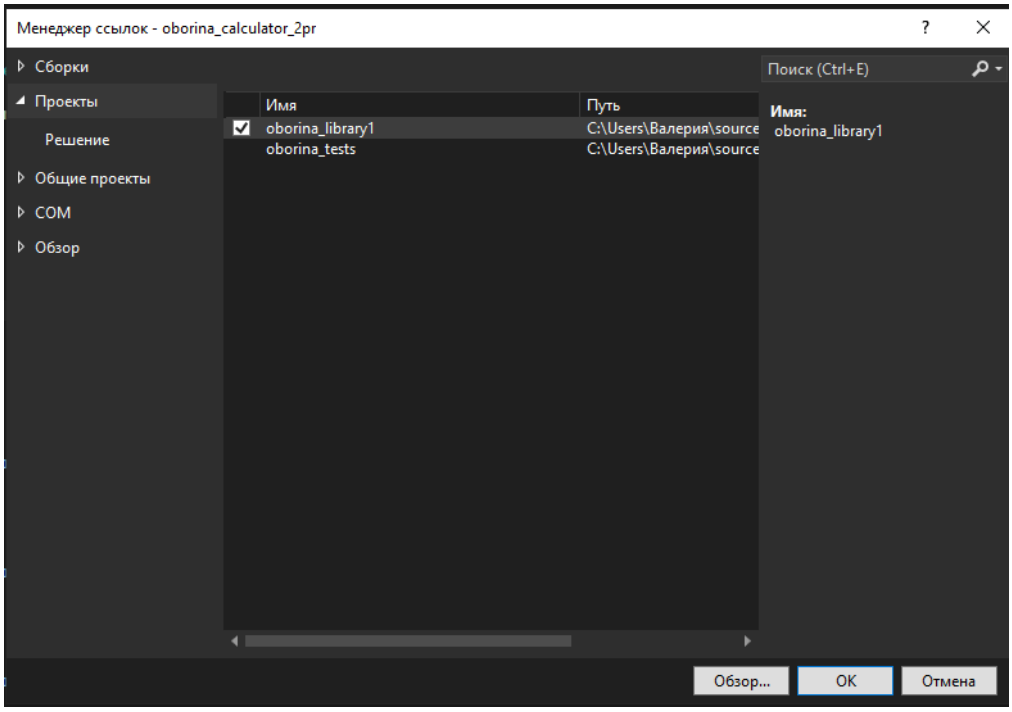

Рис.4 – Подключение библиотеки к проекту

Подключение библиотеки к основному проекту WPF – это важнейший этап сборки приложения. Так как наш интерфейс является лишь визуальной оболочкой, он не содержит в себе логики вычислений. С помощью создания *«ссылки»* мы разрешаем проекту *oborina_calculator_2pr* обращаться к методам, написанным в *oborina_library1*. Это позволяет полностью разделить дизайн и математику: если мы захотим изменить внешний вид кнопок, нам не придется трогать код расчетов, и наоборот. Такая архитектура делает программу надежной и легкой в доработке, так как каждый модуль отвечает строго за свою задачу.

Создание пользовательского интерфейса

1. Открываем файл *MainWindow.xaml*.
2. При проектировании графического интерфейса в файле MainWindow.xaml были использованы следующие настройки компонентов:
   - Для элемента *TextBox* (`tbCommand`) установлено свойство `IsReadOnly="False"`, что позволяет пользователю не только нажимать на кнопки, но и вводить выражения с клавиатуры.
   - Панель *UniformGrid* была выбрана из-за её способности автоматически распределять кнопки по сетке одинакового размера, что обеспечивает адаптивность интерфейса при изменении размера окна.
   - Свойство `TextAlignment="Right"` выбрано для текстового поля, чтобы имитировать привычный вид бухгалтерских калькуляторов *(рис.5)*

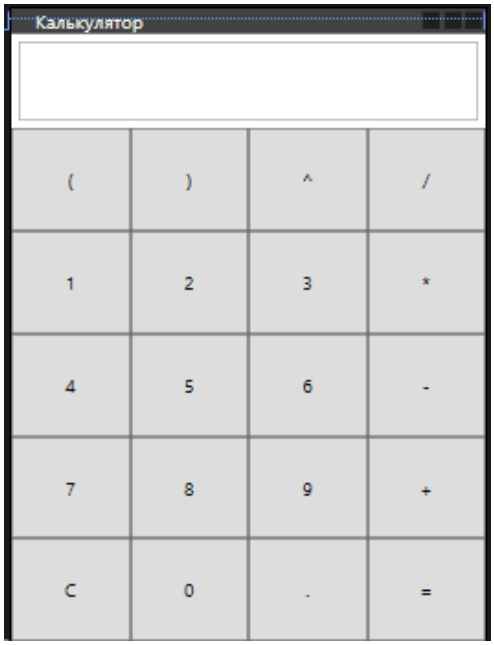

Рис.5 – Интерфейс калькулятора

3. В файле *MainWindow.xaml.cs* реализован метод `Button_Clic`. Он собирает строку выражения и вызывает метод `EvaluateExpression` из нашей библиотеки.
4. Реализована обработка ошибок: если библиотека выдает исключение (например, при делении на ноль), интерфейс выводит сообщение об ошибке *(рис.6)*.

```
        double powResult = Math.Pow(left, right);

        input = input.Substring(0, leftStart) +
                powResult.ToString(CultureInfo.InvariantCulture) +
                input.Substring(rightEnd);
    }
    for (int i = 0; i < input.Length; i++)
    {
        if (input[i] == '/')
        {
            int j = i + 1;
            while (j < input.Length && input[j] == ' ') j++;

            if (j < input.Length)
            {
                if (input[j] != '(')
                {
                    int k = j;
                    while (k < input.Length && (char.IsDigit(input[k]) || input[k] == '.')) k++;
                    string rightOperand = input.Substring(j, k - j);
                    if (double.Parse(rightOperand, CultureInfo.InvariantCulture) == 0)
                        throw new DivideByZeroException("Деление на ноль невозможно");
                }
            }
        }
    }
    var dt = new DataTable();
```

Рис.6 – логика WPF

5. На рисунке 6 представлена основная логика обработки выражения в методе `EvaluateExpression`. Поскольку стандартный вычислитель `DataTable` не поддерживает операцию возведения в степень, был реализован алгоритм предварительного разбора строки. Программа находит символ ^, определяет операнды и вычисляет результат, используя метод *Calc.Pow* из моей библиотеки *oborina_library1*.
6. Также, согласно опыту первой практической работы, реализована ручная проверка деления на ноль перед финальным вычислением. Если после знака / обнаруживается 0, программа выбрасывает исключение `DivideByZeroException` (рис. 6). Это позволяет избежать некорректных вычислений и вывести пользователю понятное сообщение об ошибке через блок `catch`.

Подключение модульного теста к библиотеке

1. Правой кнопкой по проекту *oborina_tests* — добавить — ссылка.
2. Ставим галочку на *oborina_library1*. *(рис.7)*

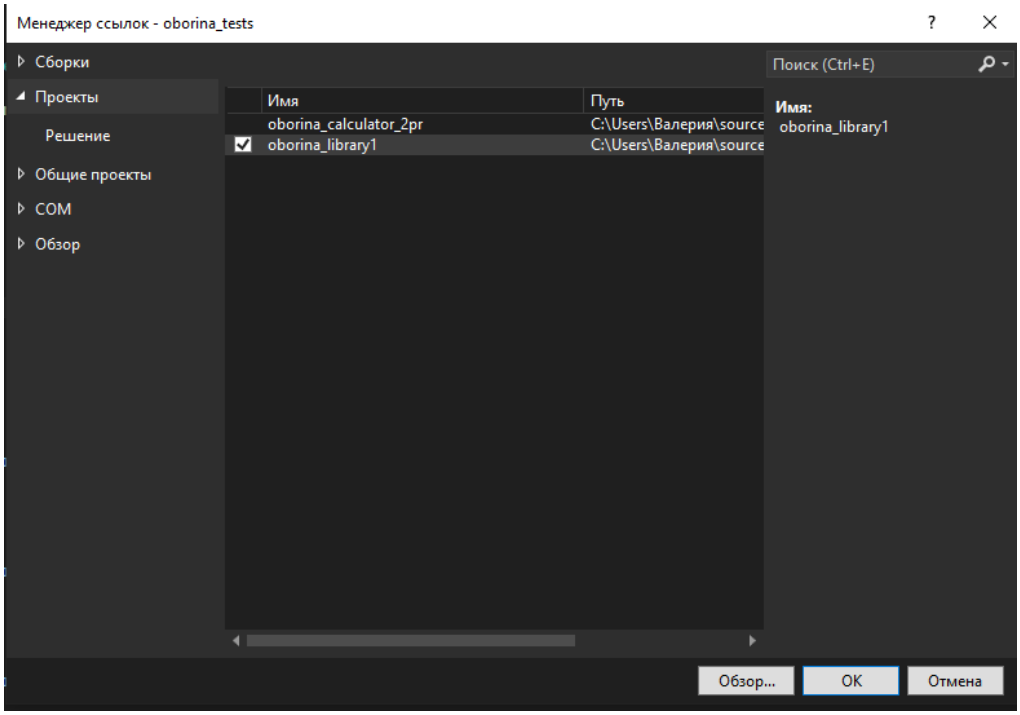

Рис.7 – Подключение библиотеки к тестам

Для верификации разработанного модуля *oborina_library1* был применен метод модульного тестирования. Основная цель тестов – убедиться, что каждый метод класса *Calc* возвращает математически верный результат. В ходе разработки использовался шаблон *AAA (Arrange, Act, Assert)*. Это позволяет четко разделить этапы подготовки данных, вызова тестируемой функции и проверки полученного результата, что делает код тестов более читаемым. Особое внимание было уделено граничным условиям, таким как работа с отрицательными числами и возведение в нулевую степень.

Код для тестов и его запуск

1. Открываем файл *UnitTest1.cs* и пишем тесты для каждой операции (сложение, вычитание, умножение, деление, степень).
2. Для проверки деления на ноль использован метод из первой практики: блок `try-catch` с проверкой сообщения через `StringAssert.Contains` *(рис.8)*.

```
ina_tests                                                    OborinaTests.CalcTests
using Microsoft.VisualStudio.TestTools.UnitTesting;
using OborinaLibrary1;
using System;

namespace OborinaTests
{
    [TestClass]
    Ссылок: 0
    public class CalcTests
    {
        [TestMethod]
        Ссылок: 0
        public void Add_ShouldReturnSum() => Assert.AreEqual(5, Calc.Add(2, 3));

        [TestMethod]
        Ссылок: 0
        public void Sub_ShouldReturnDifference() => Assert.AreEqual(2, Calc.Sub(5, 3));

        [TestMethod]
        Ссылок: 0
        public void Mul_ShouldReturnProduct() => Assert.AreEqual(20, Calc.Mul(4, 5));

        [TestMethod]
        Ссылок: 0
        public void Div_ByNonZero_ShouldReturnQuotient() => Assert.AreEqual(5, Calc.Div(10, 2));

        [TestMethod]
        Ссылок: 0
        public void Div_ByZero_ShouldThrowException()
        {
            try
            {
                Calc.Div(5, 0);
            }
            catch (DivideByZeroException ex)
            {
                StringAssert.Contains(ex.Message, "ноль");
                return;
            }
            Assert.Fail("Ожидаемое исключение не было вызвано");
        }

        [TestMethod]
        Ссылок: 0
        public void Pow_ShouldReturnPower() => Assert.AreEqual(8, Calc.Pow(2, 3));
    }
```

Рис.8 – Код модульных тестов

3. Запускаем тесты через *«Обозреватель тестов»*. Все тесты успешно пройдены *(рис.9)*.

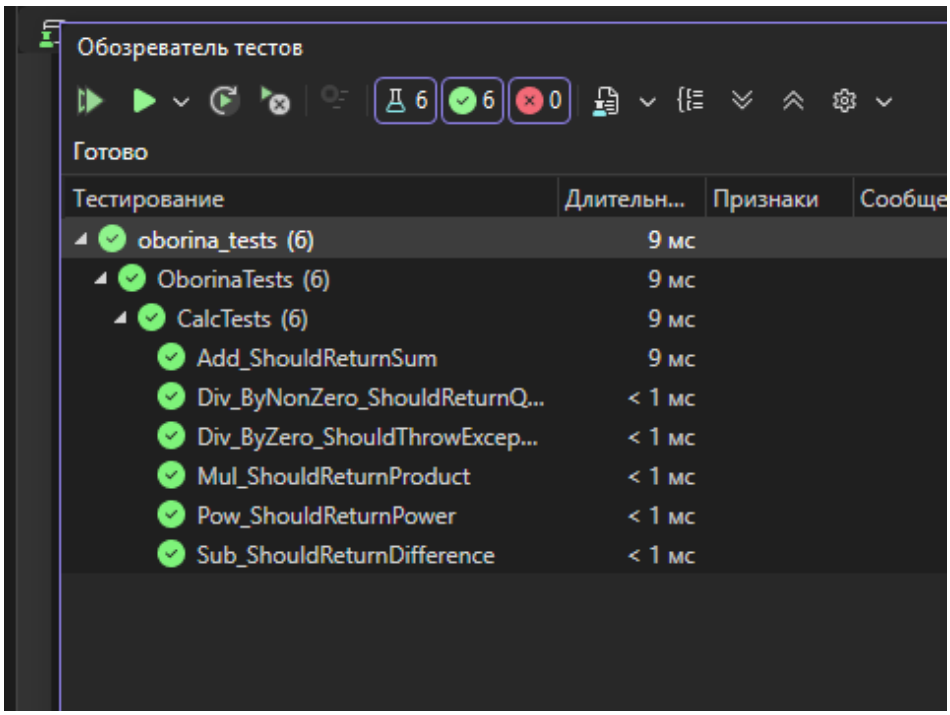

Рис.9 – Результат тестирования

На рисунке 9 видно, что все 6 разработанных тестов прошли успешно. Это подтверждает, что логика калькулятора стабильна не только в стандартных условиях, но и корректно реагирует на попытки деления на ноль, выбрасывая предусмотренное исключение. Зеленые маркеры в окне обозревателя свидетельствуют о полной готовности к интеграции в финальную версию приложения.

Заключение

Выполнение практического задания позволило освоить процесс разработки многомодульного приложения в среде Visual Studio с применением технологии WPF и инструментов модульного тестирования. В ходе работы было реализовано разделение логики приложения и его визуального представления, что обеспечило высокую гибкость и расширяемость программного продукта. Тестирование бизнес-логики на ранних этапах разработки позволило гарантировать корректность математических вычислений и надежность приложения при обработке пользовательского ввода.

Основные цели задания достигнуты

1. Создано комплексное решение, состоящее из трех независимых проектов: библиотеки классов, графического интерфейса и проекта модульных тестов.
2. Разработана библиотека классов, реализующая сложные математические вычисления, включая приоритет операций и возведение в степень, с использованием системных компонентов и рекурсивных алгоритмов.
3. Настроены взаимосвязи между модулями, что позволило реализовать принцип независимости логики от интерфейса.
4. Разработаны положительные тесты для верификации всех арифметических функций, подтвердившие точность вычислений в различных сценариях.
5. Подготовлены негативные тесты для проверки устойчивости программы к некорректным данным, в частности, реализована строгая проверка деления на ноль.
6. На основе опыта предыдущей практики, в проект модульных тестов внедрена и успешно применена методика комбинированного тестирования исключений с использованием конструкции `try`/`catch`, функций `StringAssert.Contains` и утверждения *Assert.Fail*.
7. Реализован отказоустойчивый графический интерфейс, обеспечивающий перехват исключений из библиотеки и информирование пользователя об ошибках без аварийного завершения программы.